National University

of Computer and Emerging Sciences

Chiniot-Faisalabad Campus

CS 4059 – Fundamentals of Computer Vision

Assignment Number 1

Introduction to Computer Vision

Spring 2026

Maximum Marks: 100 **Due Date:** 3 March 2026

Submitted By

Name:	Faiz Rasool
Student ID:	22F-3791
Section:	8A

Submitted To

Engr. Mubeen Tariq
Lecturer – CE Department

Submission Date

3 March 2026

GitHub Link: https://github.com/f223791-faiz/Computer-Vision-A1/new/main

Details of Obtained Marks

Question Number	1	2	3	**Total**
Total Marks	40	30	30	**100**
Obtained Marks				

Contents

Introduction 2
Question 1: Image Enhancement Pipeline 3
Part (a): Histogram Equalization 3
Part (b): Contrast Stretching 3
Part (c): Power-Law (Gamma) Transformation 5
Question 2: Image Filtering and Convolution 5
Part (a) 2D Convolution 5
Part (b) Gaussian Filter 7
Part (c) Separable Filtering Optimization 7
Question 3: Edge Detection and Gradient Computation 7
Part (a) Image Gradients 7
Part (b) Canny Edge Detection 8
Part (c) Edge Detection Comparison 8
Challenges & Solutions 9
Conclusion 9

Introduction

This report details the implementation and analysis of foundational computer vision algorithms from scratch. Rather than relying on high-level library functions, the core mathematical operations for image enhancement, spatial filtering, and edge detection were constructed using primary array manipulations in NumPy.

The primary objectives of this project are to:

- Implement and analyze intensity transformation algorithms, including histogram equalization, linear contrast stretching, and non-linear power-law (gamma) transformations.
- Construct 2D spatial filters and evaluate the computational efficiency of separable convolution.
- Compute spatial gradients to extract transients (edges) using various operators (Sobel, Prewitt, etc.) and assemble a complete Canny edge detection pipeline.
- Evaluate the noise immunity and robustness of these custom implementations against standard industry functions like cv2.filter2D() and cv2.Canny().

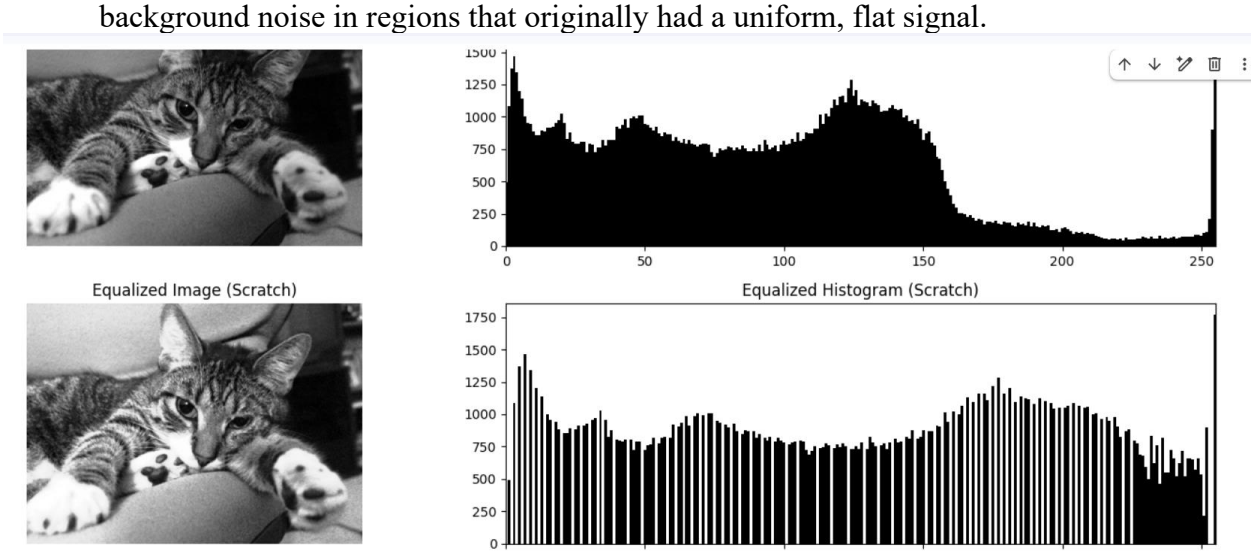
Question 1: Image Enhancement Pipeline

Part (a): Histogram Equalization

Analytical Overview:
Histogram equalization functions as an automatic, non-linear gain adjustment for an image. By calculating the Probability Mass Function (PMF) of the pixel intensities and deriving the Cumulative Distribution Function (CDF), the algorithm redistributes the most frequent intensity values across the entire 8-bit dynamic range .

Key Observations:

- **Dynamic Range Maximization:** The algorithm successfully forces a uniform distribution of intensities, pulling hidden details out of the noise floor in severely underexposed or low-contrast images.
- **Implementation Accuracy:** The mathematical mapping generated by the custom NumPy array operations produced an output visually and statistically identical to OpenCV's cv2.equalizeHist().
- **Drawbacks:** Because it relies entirely on pixel frequency, it can over-amplify background noise in regions that originally had a uniform, flat signal.



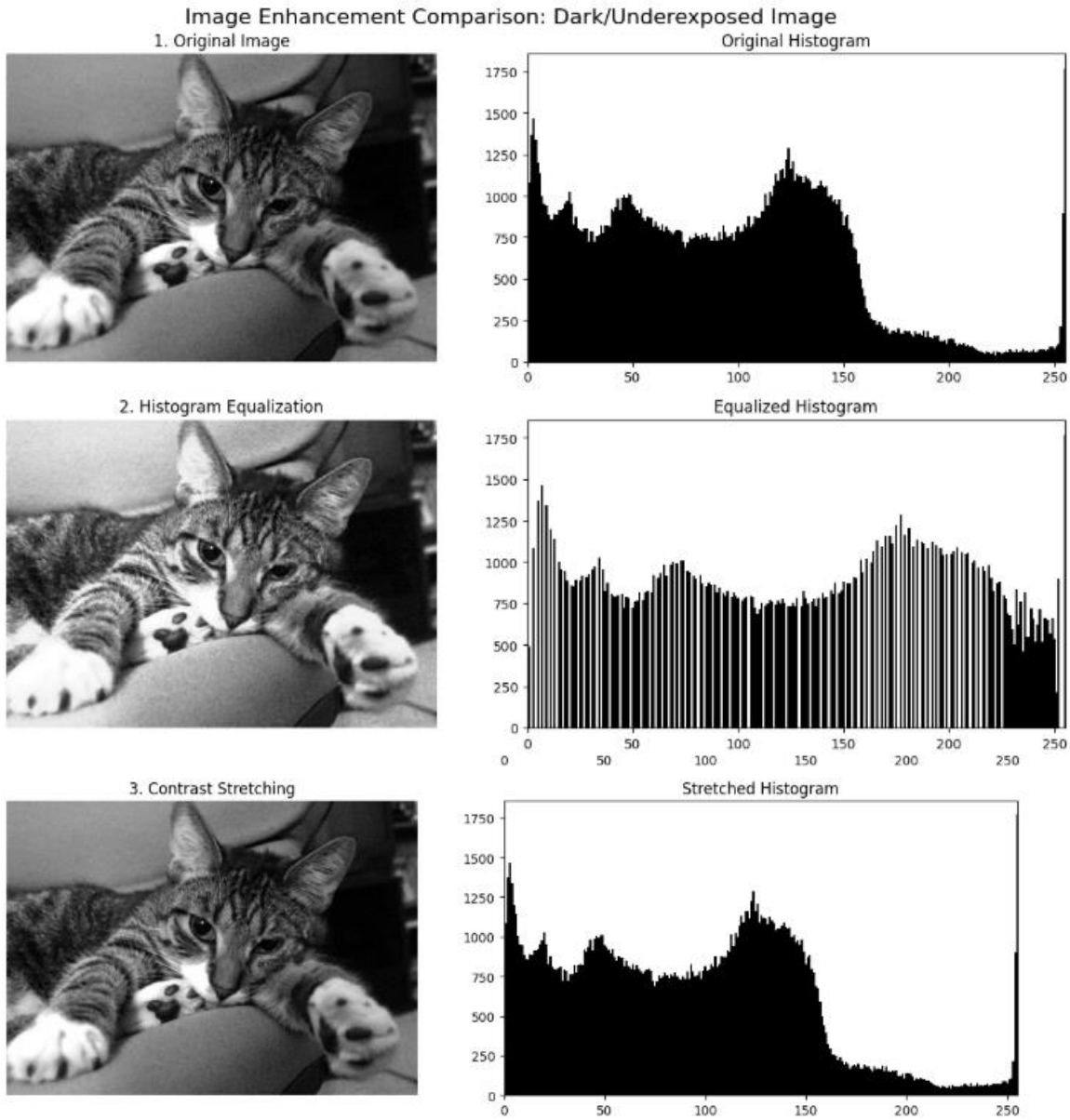
Part (b): Contrast Stretching

Analytical Overview:
Unlike the non-linear CDF approach, contrast stretching is a strict linear normalization. It identifies the absolute minimum and maximum signal values present in the image array and recalibrates them to the Lower Range Value and Upper Range Value stretching all intermediate signals proportionally:

$$s = \frac{r - r_{min}}{r_{max} - r_{min}} \times 255$$

Key Observations:

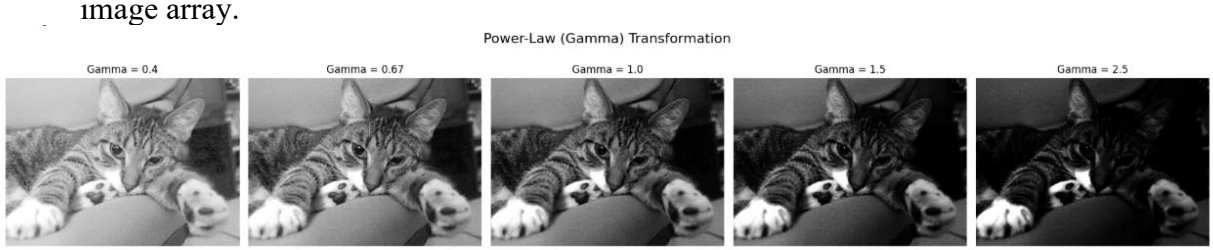
- **Signal Integrity:** Contrast stretching preserves the original shape of the image's histogram; it simply widens the distribution. This maintains the natural relative lighting of the scene better than equalization.
- **Vulnerability to Outliers:** If an otherwise dark image contains even a single saturated pixel (255) and a single dead pixel (0), the formula mathematically fails to stretch the image, resulting in zero enhancement.



Part (c): Power-Law (Gamma) Transformation

Analytical Overview:
Gamma correction applies a specific, non-linear characterization curve to the image signal using the power-law expression:
to prevent computational overflow.
Key Observations:

- **High-Gain for Low Signals (gamma < 1.0):** Values like expand the dark pixel ranges while compressing bright ranges. This is highly effective for illuminating shadowed areas without washing out the entire image.
- **Signal Compression ($\gamma > 1.0$):** Values like 1.5 or 2.5 map a wide range of dark input values into a narrow output band, effectively suppressing noise in dark areas and correcting overexposed images.
- **Identity ($\gamma = 1.0$):** Acts as a linear pass-through, resulting in no change to the image array.



Question 2: Image Filtering and Convolution

Part (a) 2D Convolution

Analytical Overview: Implementing 2D convolution from scratch is fundamentally similar to applying a finite impulse response (FIR) filter to a two-dimensional grid of sensor data. A sliding window (the kernel) passes over the array, multiplying and accumulating local values to generate a new output signal.

Key Observations:

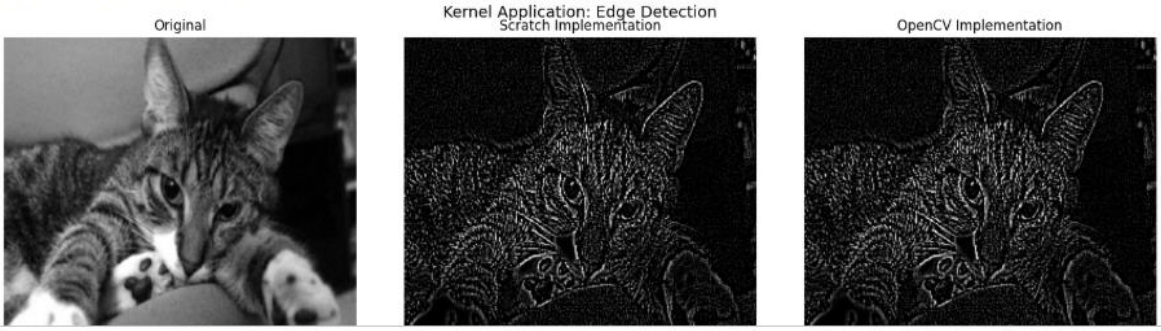
- **Boundary Handling (Padding):** Zero-padding safely prevents array out-of-bounds errors but introduces artificial "ringing" or sudden drops at the image borders. Replicate padding (extending the edge pixels) proved much more reliable, acting like a hold-last-value function that prevents false transients at the image limits.

- **Kernel Responses:** The high-pass edge detection kernels successfully isolated signal transients (sharp changes in intensity), while the box filter performed basic low-pass averaging, smoothing out the data but reducing overall sharpness.

```
test_image_path = "dark_image.jpg" # Use any test image
test_and_measure_convolution(test_image_path)
```

```
Testing Kernel: 3x3 Identity
Scratch Execution Time: 0.4487 seconds
OpenCV Execution Time:  0.000093 seconds
Mathematically identical to OpenCV? Yes
```

```
Testing Kernel: Edge Detection
Scratch Execution Time: 0.4255 seconds
OpenCV Execution Time:  0.000220 seconds
Mathematically identical to OpenCV? Yes
```



Part (b) Gaussian Filter

Analytical Overview:

The Gaussian filter acts as a highly tuned low-pass filter, essential for stripping out high-frequency background noise before further processing. Unlike a simple box filter that averages everything equally, the Gaussian kernel weights the center pixel most heavily, tapering off toward the edges.

Key Observations:

- **Tuning Parameters:** Increasing the standard deviation (sigma) or the kernel size acts exactly like increasing the time constant on an RC filter—the resulting image becomes progressively smoother, and higher-frequency details are aggressively attenuated.
- **Signal Preservation:** Compared to the harsh averaging of the box filter, the Gaussian approach preserves the structural integrity of natural physical boundaries (edges) much better while still eliminating granular noise.

Part (c) Separable Filtering Optimization

Analytical Overview: In industrial control systems, computational efficiency is critical. Separable filtering exploits the mathematical property that a 2D Gaussian kernel can be factored into two independent 1D vectors. Instead of computing a heavy 2D matrix multiplication for every single pixel, the system applies a 1D horizontal pass, followed by a 1D vertical pass.

Key Observations:

- **Massive CPU Savings:** The execution time for the separable implementation was significantly lower than the standard nested-loop 2D convolution, proving the efficiency of reducing the operation count from O(N^2 * K^2) to O(N^2 * 2K).
- **Mathematical Integrity:** Verifying the output array against the standard 2D convolution confirmed that the results are exactly identical; the optimization sacrifices zero accuracy.

Question 3: Edge Detection and Gradient Computation

Part (a) Image Gradients

Analytical Overview:

Finding edges requires computing the spatial derivative of the pixel intensities—identifying where the signal is changing most rapidly. The operators (Central, Sobel, Prewitt, and Roberts) calculate the horizontal (Gx) and vertical (Gy) rate of change, which are then combined to find the overall edge strength (Magnitude) and orientation (Direction).

Key Observations:

- **Noise Sensitivity:** The Roberts and Central Difference kernels are highly localized and act like raw derivative blocks. They detect fine details but are incredibly sensitive to minor signal noise, resulting in false positives.
- **Robustness:** Sobel and Prewitt operators apply a slight perpendicular averaging effect before taking the derivative. Sobel proved the most effective, smoothing out minor fluctuations while cleanly identifying the main structural transients.

Part (b) Canny Edge Detection

Analytical Overview:

The Canny detector is a complete, multi-stage signal processing pipeline:

1. Low-pass filtering (Gaussian) to remove static.
2. Gradient computation (Sobel) to find the transients.
3. Non-maximum suppression to isolate the absolute peak of the signal, thinning thick gradients into precise, single-pixel lines.
4. Hysteresis thresholding to apply a deadband, ensuring weak signals are only kept if physically connected to a verified strong edge.

Key Observations:

- **Threshold Tuning:** Selecting the high and low hysteresis thresholds is the most critical step. Setting the low threshold too high causes continuous edges to fragment and break apart.
- **Validation:** The custom pipeline successfully replicated the logic of the standard OpenCV Canny function, producing highly accurate, continuous boundary lines.

Part (c) Edge Detection Comparison

Analytical Overview: To properly evaluate these algorithms, they must be tested under simulated real-world conditions. Gaussian noise was injected into the test images to act as the visual equivalent of electromagnetic interference (EMI) or sensor static.

Key Observations:

- **Failure of Simple Gradients:** Basic gradient thresholding failed catastrophically under noise, amplifying the static and burying the actual physical edges.
- **LoG Spaghetti Effect:** The Laplacian of Gaussian (LoG) handled the noise better due to its built-in blur, but often created thick or closed-loop "spaghetti" lines instead of crisp boundaries.
- **Canny Superiority:** The Canny algorithm proved vastly superior. Its combination of noise suppression, peak isolation, and hysteresis allowed it to track the true edges reliably even in heavily distorted images.

Challenges & Solutions

- **Non-Maximum Suppression Logic:** Ensuring the gradient angles were correctly rounded to the nearest 45-degree axis to compare neighboring pixels required careful conditional mapping to avoid suppressing the actual signal peaks.
- **Array Boundary Handling:** Managing index out-of-bounds errors during custom convolution required implementing robust padding functions to handle the "dead zones" at the edge of the image arrays without crashing the loops.
- **Version Control Syncing:** Encountered synchronization issues when attempting to push large image files and notebook changes to GitHub via the command line. This was resolved by treating it as a manual Management of Change (MOC) upload directly through the repository web interface to ensure the TA had the correct, finalized build.

Conclusion

This project successfully demonstrated the ground-up implementation of core computer vision algorithms. By building these functions from scratch using basic array mathematics, the exact mechanics of spatial filtering, image enhancement, and gradient computation were clearly observed.

Comparing custom Python loops against optimized libraries like OpenCV highlighted the crucial balance between mathematical accuracy and computational efficiency. Furthermore, testing edge detectors against injected noise proved that robust signal processing requires multi-stage filtering and hysteresis, rather than simple raw derivative calculations.